Geometric Motion Planning

Theory and Algorithms

Travis Haran

August 28, 2026

Contents

1 Introduction

This document contains the theoretical foundations behind the Geometric Motion Planning project.

The goal of the project is to develop a motion-planning framework from first principles, beginning with fundamental computational geometry operations and gradually progressing toward classical robot motion planning, configuration-space methods, sampling-based planning, and humanoid footstep planning.

The theory presented here is developed alongside the implementation. Each major algorithm includes its mathematical foundation, geometric interpretation, computational complexity, and connection to the C++ implementation.

2 Geometric Primitives

The motion-planning system begins with a small collection of geometric primitives. These structures provide the representation used by all later geometry and planning algorithms.

2.1 Point

A two-dimensional point is represented as

$$p = (x, y), \qquad x, y \in \mathbb{R}.$$

In the implementation, coordinates are stored using double-precision floating-point values. A point is represented in C++ as:

```
struct Point {
    double x;
    double y;
};
```

2.2 Line Segment

A line segment is defined by two endpoints A and B and is denoted by $\overline{AB}$. If

$$A = (x_A, y_A), \qquad B = (x_B, y_B),$$

then the segment contains every point

$$P(t) = A + t(B - A), \qquad 0 \le t \le 1.$$

The C++ representation is:

```
struct Segment {
    Point a;
    Point b;
};
```

2.3 Polygon

A polygon is represented as an ordered sequence of vertices

$$\mathcal{P} = \{p_0, p_1, \ldots, p_{n-1}\}.$$

Each consecutive pair of vertices forms an edge, and the final vertex connects back to the first. Thus the edges are

$$\left(p_i, p_{(i+1) \bmod n}\right), \qquad 0 \le i < n.$$

This modular indexing automatically generates the closing edge $p_{n-1} \to p_0$.

For this project, obstacles are assumed to be *simple polygons* unless otherwise stated. A simple polygon has no self-intersections between non-adjacent edges.

2.4 Path

A robot path is represented using an ordered list of waypoints

$$P_0, P_1, \ldots, P_k.$$

The corresponding path segments are

$$(P_0, P_1), (P_1, P_2), \ldots, (P_{k-1}, P_k).$$

Storing waypoints rather than explicitly storing every segment avoids duplicating information and simplifies path modification.

3 Orientation and the 2D Cross Product

One of the most important predicates in computational geometry is the orientation test. Given three points

$$A = (x_A, y_A), \qquad B = (x_B, y_B), \qquad C = (x_C, y_C),$$

we want to determine whether traveling from A to B and then toward C represents a left turn, right turn, or no turn.

Define

$$\overrightarrow{AB} = B - A, \qquad \overrightarrow{AC} = C - A.$$

The two-dimensional cross product is

$$\overrightarrow{AB} \times \overrightarrow{AC} = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A).$$

We therefore define

$$\text{orientation}(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A).$$

The sign gives the geometric relationship:

$$\text{orientation}(A, B, C) \begin{cases} > 0, & \text{counterclockwise / left turn,} \\ < 0, & \text{clockwise / right turn,} \\ = 0, & \text{collinear.} \end{cases}$$

3.1 Geometric Interpretation

The magnitude of the cross product is the area of the parallelogram formed by $\overrightarrow{AB}$ and $\overrightarrow{AC}$. Therefore the area of triangle ABC is

$$\text{Area}(ABC) = \frac{1}{2} \left|\text{orientation}(A, B, C)\right|.$$

If the orientation is zero, the triangle has zero area, meaning that the three points are collinear.

3.2 Floating-Point Considerations

When coordinates are represented using floating-point values, exact comparison with zero can be unreliable. Instead, a small tolerance ε is used:

$$|x| < \varepsilon.$$

In this project,

$$\varepsilon = 10^{-9}.$$

Values sufficiently close to zero are therefore treated as zero when checking collinearity.

4 Segment Intersection

Given two line segments $\overline{AB}$ and $\overline{CD}$, we want to determine whether they intersect. This operation is fundamental to collision detection, polygon validation, visibility testing, and motion planning.

4.1 Proper Intersection

Compute

$$o_1 = \text{orientation}(A, B, C), \qquad o_2 = \text{orientation}(A, B, D),$$
$$o_3 = \text{orientation}(C, D, A), \qquad o_4 = \text{orientation}(C, D, B).$$

For a proper crossing, C and D must lie on opposite sides of the line through AB, while A and B must lie on opposite sides of the line through CD. Thus

$$o_1 o_2 < 0 \qquad \text{and} \qquad o_3 o_4 < 0.$$

Equivalently,

$$(o_1 o_2 < 0) \wedge (o_3 o_4 < 0).$$

4.2 Degenerate Cases

The proper-intersection test alone does not handle endpoint touching, collinear overlap, or one endpoint lying on another segment. These cases require a point-on-segment test.

A point P lies on $\overline{AB}$ if

$$\text{orientation}(A, B, P) = 0$$

and

$$\min(x_A, x_B) \le x_P \le \max(x_A, x_B),$$
$$\min(y_A, y_B) \le y_P \le \max(y_A, y_B).$$

In the current project convention, endpoint touching and collinear overlap both count as intersection.

4.3 Complexity

The segment-intersection test performs a constant number of arithmetic operations. Therefore,

$$T(n) = O(1).$$

Although simple, this predicate becomes one of the most frequently used operations in later motion-planning algorithms.

5 Point-in-Polygon Testing

Given a polygon $\mathcal{P}$ and query point q, we want to determine whether q lies inside the polygon. The implementation in this project uses the winding-number algorithm.

5.1 Winding Number

Imagine tracing the polygon boundary while observing how many times the polygon winds around the query point. For a simple polygon,

$$WN(q) = \begin{cases} 0, & \text{if } q \text{ is outside,} \\ +1 \text{ or } -1, & \text{if } q \text{ is inside.} \end{cases}$$

The sign depends on whether the polygon vertices are ordered counterclockwise or clockwise. The implementation therefore uses

$$WN(q) \neq 0$$

as the inside condition.

5.2 Upward Crossings

For an edge from A to B, an upward crossing occurs when

$$y_A \leq y_q \qquad \text{and} \qquad y_B > y_q.$$

If additionally

$$\text{orientation}(A, B, q) > 0,$$

the winding number is incremented:

$$WN \leftarrow WN + 1.$$

5.3 Downward Crossings

A downward crossing occurs when

$$y_B \leq y_q \qquad \text{and} \qquad y_A > y_q.$$

If

$$\text{orientation}(A, B, q) < 0,$$

then

$$WN \leftarrow WN - 1.$$

The asymmetric inequalities prevent polygon vertices lying exactly on the horizontal query line from being counted twice.

5.4 Boundary Convention

For collision detection, this project treats points lying on the polygon boundary as inside the obstacle. Therefore, before updating the winding number, each edge is checked using the point-on-segment predicate.

5.5 Complexity

Each polygon edge is examined exactly once. For a polygon containing n vertices,

$$T(n) = O(n).$$

6 Collision Checking

The geometric predicates developed so far can now be combined to answer questions relevant to robot motion.

6.1 Segment–Polygon Boundary Intersection

Given a segment s and polygon $\mathcal{P}$, the segment is tested against every polygon edge. If

$$E(\mathcal{P}) = \{e_0, e_1, \ldots, e_{n-1}\},$$

then we evaluate

$$\text{segmentsIntersect}(s, e_i)$$

for every edge e_i.

The segment intersects the polygon boundary if

$$\exists e_i \in E(\mathcal{P}) \text{ such that } s \cap e_i \neq \varnothing.$$

For a polygon containing n edges, this requires $O(n)$ segment-intersection tests.

6.2 Why Boundary Intersection Is Not Enough

Consider a segment whose two endpoints lie completely inside a polygon. The segment may never cross the polygon boundary. In that case,

$$\text{boundary intersection} = \text{false},$$

even though the segment lies entirely inside the obstacle.

This demonstrates an important distinction between *boundary intersection* and *collision*.

6.3 Collision-Free Segment

A segment is considered collision free only if, for every obstacle,

1. the segment does not intersect the obstacle boundary,
2. the first endpoint is not inside the obstacle, and
3. the second endpoint is not inside the obstacle.

Symbolically,

$$\text{CollisionFree}(s) = \neg\big(\text{BoundaryIntersection}(s) \vee \text{Inside}(s.a) \vee \text{Inside}(s.b)\big).$$

Touching an obstacle boundary is currently considered a collision.

6.4 Collision-Free Path

A path consisting of waypoints

$$P_0, P_1, \ldots, P_k$$

contains the segments

$$(P_i, P_{i+1}), \qquad 0 \leq i < k.$$

The path is collision free if every one of these segments is collision free:

$$\text{CollisionFreePath}(\mathcal{P}) = \bigwedge_{i=0}^{k-1} \text{CollisionFree}\big(\overline{P_i P_{i+1}}\big)\,.$$

This creates the first complete geometric motion-validity test in the project. Later planners will repeatedly call these collision predicates when constructing visibility graphs, PRMs, RRTs, and humanoid footstep transitions.